Baseline Retrieval vs. Cross-Encoder Re-Ranking: A Two-Stage RAG Study on the Gloomhaven Rulebook

Sampath Ravikanti

October 2026

1 Setup

Corpus	Gloomhaven rules reference, 74,916 characters
Sectioning	39 headed sections detected from the layout
Chunking	550 characters, section-aware; 167 chunks
Chunk sizes	min 160 / median 490 / max 571 characters
Bi-encoder	`all-MiniLM-L6-v2` (384-dim, local)
Cross-encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`
Generator	`gemini-3.8-flash`
Vector store	normalised matrix, cosine via one NumPy matmul

Both pipelines share stage-1 retrieval and call the same generation function with an identical prompt, so the only thing that differs between Answer 1 and Answer 2 is which three chunks go into the context.

I picked a rulebook over a policy or legal PDF on purpose. The failure modes the brief asks for – keyword traps, exceptions, negation – need a document that defines a lot of near-synonymous terms and then qualifies them. Gloomhaven keeps *exhausted* separate from *eliminated* and *lost* separate from *discarded*, and it usually states a general rule in one place and the restriction on it somewhere else. A regulatory document does the same thing, but over far more text for the same number of usable test cases.

2 Architecture Comparison

The difference is where in the computation the query meets the document.

A bi-encoder runs each text through the transformer on its own and mean-pools the token states into one vector. A chunk vector depends on nothing but the chunk, so it can be built once at index time; the query is embedded at run time and compared by cosine similarity. The whole interaction between query and chunk is one dot product between two vectors that were produced without seeing each other. Anything pooling threw away is gone by scoring time, and pooling throws things away in a particular way – it ignores order and is dominated by the heavy content words. The little words that flip the meaning (*not*, *cannot*, *except*) barely move the vector, which is why "a stunned figure may use items" and "a stunned figure cannot use items" end up as near neighbours.

A cross-encoder never produces comparable vectors at all. It concatenates query and chunk into one sequence and runs self-attention across the pair, so every query token can attend to every chunk token, and the output is a single score for that one pairing. That is what lets it react to exact token alignment: *cannot* in the chunk can attend to *may* in the query and drag the score down. Once both sides have been pooled, that is no longer possible.

What it costs is precomputation. A bi-encoder index is built once; a cross-encoder score only exists after the query arrives, and has to be recomputed for every candidate. That asymmetry is the whole reason for the two-stage design, and Section 6 puts numbers on it.

3 Query Set and Method

Eleven queries, written against rules I found by reading the document first and then phrased in everyday synonyms rather than the rulebook's own vocabulary ("strike back" for *retaliate*, "shut entryway" for *closed door*). For each query I marked a gold chunk by hand, and a query counts as answered only if the gold chunk is in the top 3.

Phrasing turned out to matter more than I expected. If the queries reuse the document's own words, lexical overlap alone puts the right chunk first, both pipelines succeed and the experiment measures nothing. Writing "strike back" instead of *retaliate* takes that shortcut away and forces retrieval to work semantically, which is the only condition under which the two architectures actually behave differently. I also wrote each query after locating a genuinely awkward rule, rather than guessing at queries and hoping some would fail.

4 Qualitative and Rank-Shift Analysis

4.1 Outcome across the query set

Verdict	bi → ce	Query
Fixed by re-rank	7 → 1	am I allowed to skip part of what my card tells me to do?
Fixed by re-rank	6 → 2	may a stunned figure drink a healing potion on its turn?
Fixed by re-rank	7 → 3	what occurs if a monster has nothing it is able to aim at?
Fixed by re-rank	9 → 2	can an enemy creature grab the treasure chest in the room?
Artifact (see 4.5)	4 → 3	may I stockpile two of the same piece of equipment?
Both correct	1 → 1	does mending someone's injuries also cure the venom?
Both correct	2 → 1	can a beast walk through a shut entryway?
Broken by re-rank	2 → 6	do I get a bigger hand of cards as I grow more powerful?
Broken by re-rank	2 → 4	do sealed entrances open when someone steps onto them?
Stage-1 miss	—	if I strike back at someone who hit me, does that count as attacking?
Stage-1 miss	—	is there a ceiling on restoring a wounded hero's vitality?

So: four genuine rescues, two rank regressions (neither of which changed the answer – see 4.3), and two failures re-ranking could not touch because the gold chunk never reached it.

4.2 Rank shifts on the rescued queries

Query	Gold chunk	bi-rank	ce-rank	bi-score	ce-score
skip part of an action	17	7	1	0.330	−3.475
stunned figure, potion	43	6	2	0.344	−1.680
monster with no target	71	7	3	0.476	−1.628
monsters looting treasure	80	9	2	0.285	−5.546

The bi-encoder scores are the interesting column. On the first query the gold chunk scored 0.330 against 0.396 for the top distractor – 0.066 separating rank 1 from rank 7, with all ten candidates inside a band about 0.07 wide. When every chunk on the topic gets roughly the same similarity, the ordering inside that band is decided by differences that do not correspond to anything about relevance.

That is what mean-pooled cosine similarity does. Pooling maps any passage about taking a turn into much the same region of the space, so chunks that share a topic share a score, and there is no resolution left to separate the one chunk that answers the question from six that merely talk around it. The cross-encoder scores on the same candidates run from -3.5 to -11.3 – several points apart instead of hundredths. Because it scores each pair jointly it is not obliged to keep all the turn-related chunks together, and it pulls them apart.

4.3 Answer 1 vs Answer 2

On the stun query the baseline did not hallucinate. It correctly reported that it could not answer:

> **Answer 1 (baseline):** "the answer is not contained within the text… the context does not mention the 'stunned' condition or rules regarding drinking potions."
>
> **Answer 2 (re-ranked):** "No, a stunned figure cannot drink a healing potion on its turn. *STUN: The figure cannot perform any abilities or use items except, if a character, to perform a long rest.*" (chunk 43)

That failure mode is worth dwelling on, because it is not the one the brief expects. The baseline worked out that its three chunks did not cover the stun condition and said so. That comes from the prompt, which tells the model to state when the context is insufficient instead of guessing. Under that instruction, a retrieval failure degrades into an abstention rather than a fabrication, which is the safer outcome – a user told "this isn't in the context" knows to go and look, a user given a confident wrong answer does not.

I would not conclude that the prompt solves anything. The same three chunks under a looser prompt, or with a model more willing to fill the gap, would plausibly have produced an answer stitched together from the general action rules: fluent, specific and wrong. And abstention is still a failure. The user asked an answerable question about a rule that is in the document and got nothing back. The prompt turned a wrong answer into a missing one. Retrieval quality sets a ceiling on answer quality that prompt wording can protect but cannot raise.

The exceptions query shows the same contrast in a milder form. The baseline had the general permission and knew it was not enough:

> **Answer 1 (baseline):** "it is not explicitly stated whether an enemy creature can grab a treasure chest. The context only specifies that: *Treasure tiles can be looted by a player.*" (chunk 92)
>
> **Answer 2 (re-ranked):** "No, an enemy creature cannot grab or loot a treasure tile. *Monsters are unable to loot treasure tiles.*" (chunk 80)

The two regressions are more interesting than I expected, because neither of them survives contact with the generated answers. Both involve a general rule and a specific instance of it. On the hand-size query the cross-encoder promoted chunks about levelling up and card availability over chunk 125, which states that levelling up never changes maximum hand size. That is a real rank regression – but the answers are both correct. Answer 1 quoted chunks 125 and 127; Answer 2 lost 125 and quoted 127 instead, which says the hand limit "must remain the same" and answers the question just as well. On the sealed-entrances query both pipelines abstained, so re-ranking cost nothing there either.

So the honest statement is that re-ranking demoted the gold chunk twice and degraded the answer zero times. That matters for how the retrieval metric should be read: ranking the marked chunk out of the top 3 is only a failure if no other retrieved chunk supports the same answer, and on this document rules are often stated in more than one place. The mechanism

behind the demotions still looks real – in both cases the demoted chunk answers in a single clause while the promoted ones discuss the topic at length, and a re-ranker trained on web passage relevance appears to reward elaboration on the query's subject. I would expect that to do visible damage on a corpus that states each rule exactly once. It simply did not do any here.

4.4 Failure categories covered

Spec category	Query	Evidence
Keyword trap / distractor	am I allowed to skip part of what my card tells me to do?	Baseline top 3 (chunks 18, 11, 16) are saturated with *card*, *action*, *turn*; the gold chunk 17 phrases the rule as "free to not perform any part". Rank 7 → 1.
Exceptions & corner cases	can an enemy creature grab the treasure chest in the room?	Baseline returns the general looting rules (chunk 92, "treasure tiles can be looted by a player"; chunk 61, end-of-turn looting). The restriction sits in the monster section: "Monsters are unable to loot treasure tiles" (chunk 80). Rank 9 → 2.
Negation & constraints	may a stunned figure drink a healing potion on its turn?	Gold rule is stated as *cannot... except*; baseline abstains, re-rank quotes it. Rank 6 → 2.
Buried chunk (ranks 4–10 promoted)	what occurs if a monster has nothing it is able to aim at?	Gold chunk 71 at rank 7, promoted to 3. Also observed at ranks 6, 9 and 4 on the other queries.

The treasure query is the cleanest example of the exceptions case in the whole set. Both halves of the rule are about looting, so the bi-encoder has no way to tell that one of them is the general permission and the other is the restriction that governs the question; it ranked the general statements first and left the restriction at rank 9. The cross-encoder, reading "enemy creature" against "monsters are unable to..." in the same forward pass, moved it to rank 2.

4.5 A measurement artifact worth reporting

The duplicate-items query shows up as fixed, but it is not. The rulebook states the rule twice, in different words:

"You cannot own more than 1 copy of any item card."
"No character may own duplicates of the same item."

My gold substring only matched the first. The baseline's top 3 contained the second, so it answered correctly and was scored wrong.

Any evaluation that tests for a fixed substring has this problem, and the bias is not even-handed. A rule stated once in the gold string's words is scored correctly; a document that paraphrases itself produces false failures for whichever pipeline happened to retrieve the paraphrase. Since I built the query set around cases where the baseline was expected to struggle, errors of this kind flatter the re-ranker. There are two fixes: allow a set of acceptable phrasings per query, or stop using chunk identity as the criterion and judge the generated answer against the rule instead. The second is more work and more honest, because retrieving a different chunk that supports the same correct answer is not a retrieval failure in any sense a user would recognise.

4.6 Sensitivity to chunk size

Chunk size	Chunks	Fixed	Broken	Stage-1 misses
450	219	2	0	3
550	167	5	2	2
700	126	2	1	2

This is the caveat I would keep if I could keep only one, because it bounds the headline result quite hard.

At 700 characters most of the measured benefit disappears – two rescues instead of five. The reason is mechanical: bigger chunks are more likely to contain both the general rule and the exception that qualifies it, so the baseline retrieves a passage that happens to include the answer and stops failing. There is simply less left for the re-ranker to repair. At 450 the opposite happens and stage-1 misses go up to three, because rules get split across chunk boundaries and no single chunk holds enough context to score well.

Which means the benefit reported here is partly a function of a chunking parameter I chose. Another reasonable choice would have produced a visibly different conclusion. Re-ranking pays off in a specific regime – chunks small enough to isolate a rule from its exception, and a corpus where lots of chunks are topically similar. Both hold here at 550 characters; neither is a general law. Reporting only the 550 row would be using a tuned number to support an untuned claim.

5 Latency Profiling

	Baseline	With re-ranking	Overhead
Mean over 11 queries	7.0 ms	160.6 ms	22.8×
Range	6.3–10.0 ms	132.5–178.6 ms	13.8–26.6×

Measured on CPU. Stage-1 cost is one query embedding (10.8 ms) plus a 167×384 matmul. The cross-encoder costs 13.2 ms per (query, chunk) pair.

A 22.8× overhead sounds bad and is close to irrelevant here. The absolute numbers are 7.0 ms and 160.6 ms, and both of them feed an LLM call that takes one to three seconds. Re-ranking adds about 150 ms to a pipeline whose latency is dominated by something an order of magnitude larger. The multiplier only alarms you if you read it without units: it is comparing a very fast operation to a slightly less fast one. Where re-ranking does start to hurt is throughput under concurrent load, because those 150 ms are real CPU work holding the core, which the matmul is not.

6 Scaling: Why Not Cross-Encode Everything?

At a measured 13.2 ms per pair:

Scope	Time per query
Top-10 re-rank (what we do)	0.13 s
This corpus, all 167 chunks	2.20 s
A 100k-chunk corpus	1,315.31 s (22 min)
A 1M-chunk corpus	13,153.14 s (3.7 hours)

The answer falls out of where the query enters the computation.

For the bi-encoder, chunk vectors depend only on chunks. They are computed once when the index is built and reused unchanged by every query after that, so the per-query work is one embedding plus a similarity search – and an ANN index makes that sublinear in corpus size. Adding a million chunks costs build time and memory but barely changes query latency.

For the cross-encoder the score is a property of the pair. Nothing about a chunk's relevance can be known before the query exists, so no work amortises across queries and cost is strictly linear in the number of candidates scored. The 13.2 ms per pair makes that concrete: scoring this 167-chunk corpus exhaustively takes 2.2 seconds per query, and 100k chunks would take 22 minutes. That is not just slow, it is the wrong shape of computation to run at corpus scale.

Two-stage retrieval gets around the asymmetry by letting each model do the thing it is cheap at. The bi-encoder cuts the corpus down to 10 candidates at index-time cost, and the cross-encoder settles those 10 in 0.13 seconds – roughly cross-encoder precision at roughly bi-encoder cost.

The arrangement has a hard limit, and this data shows it. The re-ranker can only reorder what stage 1 handed it. Two queries failed in both pipelines because the gold chunk never made the top 10 at all – the retaliate and heal-cap queries, both vocabulary mismatches that share almost no content words with the rule that governs them. No amount of re-ranker quality recovers those. Stage-1 recall is the ceiling; stage 2 only decides how much of it you get.

7 Conclusion

The experiment compared two pipelines that differ in exactly one step, holding corpus, chunking, prompt and generator constant. Across eleven queries, cross-encoder re-ranking produced four genuine rescues, two rank regressions that did not damage either answer, and left two failures untouched because the gold chunk never reached the candidate set. A fifth apparent rescue turned out to be an artifact of substring scoring.

Four out of eleven is a real effect but a modest one, and the chunk-size sweep bounds it: the benefit is largely tied to a chunk size that separates rules from their exceptions, and at 700 characters it mostly goes away. The regressions turned out to be the weaker half of my own argument. They are not random – they come from the same preference for elaboration that produces the rescues – but on this corpus both demoted chunks had a near-duplicate still in the context, so neither answer suffered. On a document that states each rule once I would expect them to cost something real.

What comes through most clearly is how compressed the bi-encoder scores are. Ten candidates inside a 0.07 band, with the correct chunk at rank 7 and 0.066 behind a distractor, is not a near miss – it says cosine similarity over pooled embeddings had no resolution left to make that distinction. The cross-encoder spread the same candidates over several points. That is the mechanism behind every rescue here, and the concrete form of the architectural difference in Section 2.

With more time I would work on stage 1 rather than stage 2. Both unreachable failures were vocabulary mismatches, which is exactly what a lexical signal is good at; a hybrid BM25 plus

dense first stage would probably have put both gold chunks in the top 10 and handed them to a re-ranker that has already shown it can promote them. Raising stage-1 recall lifts the ceiling. More work on stage 2 only redistributes what is underneath it.

A Reproducing

```
curl -sL https://www.orderofgamers.com/downloads/Gloomhaven_v1.1.pdf \
     -o gloomhaven_rulebook.pdf
export GEMINI_API_KEY="..."
jupyter notebook rag_rerank.ipynb
```

Code: `https://github.com/Sampath7099/RAG-assignment`